HBOS · 裸机内核操作系统

HBOS 应用开发手册

HAX Application Development Guide · .hax / C 版

本手册随 HBOS 内核源码发布，供第三方应用开发者参考。

HBOS APPLICATION DEVELOPER GUIDE

2026/08/16 · VERSION 1.9 · HBOS v0.1-beta5 · HIVE 0.1-beta5-gui.4（Toolkit API 1.4 / TUI Kit 1.0）

目录

CONTENTS

第 1 章 — 概述、格式与类型系统

1.1 关于本手册
1.2 什么是 HAX 应用（.hax）
1.2.1 ELF64 结构
1.2.2 .haxmeta 元数据段
1.3 自动导入机制
1.3.1 构建流水线详解
1.3.2 预编译应用的导入
1.4 编译 HAX 应用
1.4.1 Makefile 自动构建
1.4.2 手动编译步骤
1.4.3 链接脚本与地址空间
1.5 专有类型系统
1.6 应用元数据宏 HAX_APP()

第 2 章 — HAX SDK 参考手册

2.1 文本输入输出
2.2 文件操作
2.3 系统服务
2.4 错误处理约定
2.5 直接使用 POSIX libc
2.6 目录遍历（opendir / readdir）
2.7 GUI 全屏画布接口
2.8 并发窗口接口（推荐）
2.9 HIVE 标准窗口控件 API
2.10 TUI 终端控件 API

第 3 章 — 运行、发现与调试

3.1 TUI 与 GUI 类型说明
3.2 apps / run 命令
3.3 在图形桌面运行
3.4 参数传递
3.5 退出码约定

第 4 章 — 完整示例

4.1 最小应用（TUI，文档示例）
4.2 输入循环（GUI，文档示例）
4.3 catf —— 读取并显示文件
4.4 ls —— 列出目录内容
4.5 GUI 全屏画布（文档示例）
4.6 HIVE 并发窗口（文档示例）
4.7 HIVE Toolkit API 1.4 控件（文档示例）
4.8 实机运行画面（QEMU 截图）

附录 A — 底层 POSIX 系统调用速查

附录 B — hax_meta_t 二进制布局

附录 C — 常见构建错误排查

第 1 章

OVERVIEW, FORMAT & TYPES

本章介绍 HAX 应用格式的设计目标、内部结构、自动导入机制、编译工具链以及类型系统。

1.1 关于本手册

本手册面向希望在 **HBOS**（裸机内核操作系统）上开发应用的用户。HBOS 提供一套名为 **HAX**（HBOS Application eXecutable）的应用机制，设计目标是：

- **零注册**——把源码（`.c`）或预编译应用（`.hax`）放进 `./app` 目录，执行一次 `make`，应用在系统启动后自动出现，无需修改内核代码或配置文件。
- **标准入口**——应用写 `main(argc, argv)`，与普通 C 程序没有区别。
- **轻量 SDK**——一个头文件 `<hax.h>` 覆盖 90% 的常用场景；更底层的需求可直接调用 POSIX libc。
- **可移植**—— `.hax` 是标准 ELF64，可用任何支持 x86-64 裸机 freestanding 的工具链构建。

> 本手册中所有命令示例默认你已在 HBOS 系统提示符（`HBOS>`）或 HBOS 构建机（宿主 Linux/x86-64）下操作。

1.2 什么是 HAX 应用（.hax）

1.2.1 ELF64 结构

一个 `.hax` 文件就是标准 **ELF64 可执行程序**——ELF Magic、Program Headers、.text/.data/.bss 段均与普通用户态 ELF 完全相同，可用 `readelf -h foo.hax` 验证。它以静态链接方式包含 HBOS 用户态 libc 与 crt0，运行时加载到地址 `0x1000000000`（256 GiB），通过 `int 0x80` 陷入内核进行系统调用。

1.2.2 .haxmeta 元数据段

与普通 ELF 的唯一区别是额外携带一个名为 `.haxmeta` 的只读段，其中存放一个 `hax_meta_t` 结构（见 1.6）。该段仅供 `tools/genhax.py` 在构建时读取，内核在运行时通过 blob 清单而非 ELF 段头来定位元数据，因此此段不占用运行时内存。

扩展名 `.hax` 是构建系统的识别标志，**与其他规范中的 .epf/.elf 等扩展名无关**。

1.3 自动导入机制

1.3.1 构建流水线详解

核心逻辑：**只要 `./app` 目录里存在 `.hax` 文件（编译产物或预编译），它就会被自动加入系统。**

步骤	触发条件	具体行为
① 编译	`app/*.c` 有更新	Makefile 用 HAX 工具链把每个 `app/foo.c` 编译为 `build/app/foo.hax`（ELF64，含 `.haxmeta`）
② 解析	`build/app/*.hax` 有更新	`tools/genhax.py` 依次解析每个 ELF 的 `.haxmeta` 段（magic 校验 → 提取 kind/name/desc），若无合法元数据则以文件名兜底（kind=TUI）
③ 打包	同上	生成器按 16 字节对齐拼接所有 ELF 为 `build/hax_blob.bin`，并输出 `build/hax_manifest.c`（含 `hax_app_table[]` + 计数）
④ 嵌入	blob 有更新	`src/user/hax_blob.asm` 用 `incbin` 把 blob 嵌入内核只读数据段；manifest 编译进内核
⑤ 注册	系统启动	内核通过 `hax_app_table[]` 查表；`apps` 命令列出，`run <名>` 从 blob 切片后 ELF 加载并生成任务

> 步骤 ②③ 由同一次 Python 脚本完成（GNU Make `&:` grouped target），blob 与 manifest 原子更新，不会出现两者不一致的情况。需 GNU Make ≥ 4.3（Ubuntu 22.04+ 默认满足）。

1.3.2 预编译应用的导入

你可以把别处交叉编译好的 `.hax`（只需是合法 ELF64 + `.haxmeta`，构建主机不限）直接放进 `./app`。执行 `make` 时，生成器会与 `./app/*.c` 产物一起打包。预编译文件无需改动 Makefile，也无需源码。

1.4 编译 HAX 应用

1.4.1 Makefile 自动构建

```
make                # 构建整个系统，含所有 .hax 自动打包
make hax-apps       # 仅构建 ./app 下的应用，不重新链接内核
```

把源文件加入 `./app` 后不需要改任何 Makefile——通配符规则会自动发现新文件。

1.4.2 手动编译步骤

Makefile 内部等价命令如下，可用于调试或在其他构建系统中集成：

```
# 1. 编译目标文件
gcc -c -m64 -ffreestanding -fno-stack-protector -fno-pie \
    -mno-red-zone -mno-sse -mno-sse2 \
    -Iapp/include -Isrc/user -Isrc/user/libc \
    app/myapp.c -o build/app/myapp.o

# 2. 链接为 .hax（ELF64 静态）
ld -m elf_x86_64 -static -nostdlib -T src/user/user.ld \
   build/user/libc/*.o build/user/crt0.o \
   build/app/myapp.o -o build/app/myapp.hax

# 3. 打包（可选，单独重新打包）
python3 tools/genhax.py \
   --blob  build/hax_blob.bin \
   --manifest build/hax_manifest.c \
   build/app/myapp.hax
```

1.4.3 链接脚本与地址空间

所有 HAX 应用共享链接脚本 `src/user/user.ld`，加载基地址固定为 `0x1000000000`（256 GiB）。内核在执行 `elf64_load_and_spawn` 时把各段映射到用户页表的对应位置；每个应用运行在独立地址空间（独立任务），互不干扰。

地址范围	用途
`0x1000000000+`	用户态代码与数据（.text / .data / .bss）
栈顶（由内核分配）	用户栈（初始 128 KiB）

0x0000 – 0x0FFF	保留（NULL 陷阱页，访问触发 #PF 而非挂起）

1.5 专有类型系统

<hax.h> 定义了一组与平台无关的定宽类型，在 32/64 位工具链上都有相同语义：

类型	位宽	有符号	C 底层类型（x86-64）	典型用途
HI8	8	是	signed char	字节序列、字符
HU8	8	否	unsigned char	原始字节、颜色分量
HI16	16	是	short	小范围整数
HU16	16	否	unsigned short	端口号、UTF-16 单元
HI32	32	是	int	通用整数、返回值
HU32	32	否	unsigned int	颜色值 0xRRGGBB
HI64	64	是	long long	文件偏移、大整数
HU64	64	否	unsigned long long	位掩码、地址
HCOLOR	32	否	unsigned int	RGB 颜色（0x00RRGGBB）
hax_meta_t	—	—	struct 104 字节	应用元数据（构建时用）

1.6 应用元数据宏 HAX_APP()

在源文件**全局作用域**（任意位置，不得在函数内）调用一次 HAX_APP() 宏，声明该应用的名称、描述和类型：

```
HAX_APP("myapp", "我的第一个 HBOS 应用", HAX_KIND_TUI);
```

宏展开后会把一个 hax_meta_t 常量放进 .haxmeta ELF 段，标记为 __attribute__((used)) 防止链接器优化掉。

参数	类型	约束	说明
name	字符串字面量	≤ 31 字节，UTF-8	应用唯一标识符，run 命令用此名查找；建议全小写英文，无空格
desc	字符串字面量	≤ 63 字节，UTF-8	一句话描述，显示在 apps 列表中
kind	常量	见下表	应用类型，影响启动器中的分类标签

kind 常量	值	含义
HAX_KIND_TUI	1	终端文本界面应用（命令行运行）
HAX_KIND_GUI	2	图形桌面应用（出现在桌面启动器）
HAX_KIND_BOTH	3	两种入口均注册
HAX_KIND_GUI_WIN	4（标志位）	与 GUI kind 按位 OR，声明应用使用可并发窗口、应非阻塞启动

> 每个 .hax 文件只能有一个 HAX_APP()。若检测到多个 .haxmeta 符号，构建将报错。

第 2 章

HAX SDK REFERENCE

SDK 是对 HBOS 用户态 libc 与 int 0x80 系统调用的轻量包装，让常见操作一行搞定。所有声明在 <hax.h>，无需额外链接标志。

2.1 文本输入输出

void hax_print(const char *s);

把字符串 s 写到标准输出，**不**追加换行。s 为 NULL 时行为未定义。

void hax_println(const char *s);

输出 s 后追加一个换行符（\n）。等价于 hax_print(s); hax_print("\n");

void hax_printf(const char *fmt, ...);

格式化输出，语义同 C99 printf。支持格式说明符：

说明符	类型	输出
%d / %i	int	有符号十进制
%u	unsigned int	无符号十进制
%x / %X	unsigned int	十六进制（小写/大写）
%ld / %lu	long / unsigned long	64 位十进制
%s	char *	字符串
%c	int	单字符
%p	void *	指针（十六进制，0x 前缀）

%%	—	字面量 %

```
hax_printf("进程 PID=%d，名称=%s\n", hax_pid(), "myapp");
hax_printf("地址 0x%p，计数=%lu\n", ptr, count);
```

int hax_input(char *buf, int cap);

从标准输入读取一行到 buf（含 NUL 终止符，**不含**结尾换行）。

- 最多写入 cap - 1 个有效字符，第 cap 字节写 '\0'。
- 返回实际字符数（不含 NUL）；遇到 EOF 或错误返回 -1。
- buf 为 NULL 或 cap <= 0 时直接返回 -1。

```
char line[128];
hax_print("请输入指令: ");
int n = hax_input(line, sizeof(line));
if (n < 0) { hax_println("读取失败"); hax_exit(1); }
hax_printf("你输入了 %d 个字符: %s\n", n, line);
```

int hax_getch(void);

读取并返回一个字节（0–255）；无数据时返回 -1。该函数不回显字符、不等待换行，适合逐键处理。

```
hax_println("按任意键继续...");
while (hax_getch() < 0) hax_sleep(0);   /* 轮询 */
```

2.2 文件操作

long hax_read_file(const char *path, void *buf, long cap);

打开 path，读取全部内容到 buf（最多 cap 字节），关闭文件。

- 返回实际读到的字节数（文件长度 ≤ cap 时等于文件大小）。
- 失败（文件不存在、权限不足、路径为 NULL）返回 -1。
- 不追加 NUL，若需当字符串使用，请自行在返回值处置 '\0'。

```
char data[4096];
long n = hax_read_file("/etc/hostname", data, sizeof(data) - 1);
if (n >= 0) { data[n] = '\0'; hax_printf("主机名: %s\n", data); }
```

long hax_write_file(const char *path, const void *buf, long len);

以覆盖创建模式打开 path，把 buf 的 len 字节写入，关闭文件。

- 返回实际写入字节数；失败返回 -1。
- 若文件不存在则创建；若已存在则截断后写入。

```
const char *msg = "Hello, HBOS!\n";
long w = hax_write_file("/tmp/test.txt", msg, 13);
hax_printf("写入 %ld 字节\n", w);
```

2.3 系统服务

函数签名	返回值	说明
void hax_sleep(unsigned sec);	无	休眠整数秒。sec=0 为 yield（让出 CPU，不挂起）
void hax_exit(int code);	不返回	以状态码终止当前应用。code=0 表示成功，非零表示错误
int hax_pid(void);	进程 ID	返回当前任务的内核 PID（≥1 的正整数）

2.4 错误处理约定

SDK 函数遵循以下约定，与 POSIX libc 一致：

- 返回 int 或 long 的函数，失败时返回 **-1**（不设 errno，HBOS 当前不暴露全局 errno）。
- 返回指针的函数，失败时返回 **NULL**。
- hax_exit / hax_sleep 不会失败。

> HBOS v0.1 暂不设 errno。若需区分具体错误原因，请使用底层 open/read/write 系统调用（见附录 A），它们通过返回值编码错误码（-ENOENT、-EACCES 等）。

2.5 直接使用 POSIX libc

SDK 是轻量封装，满足不了的场景可直接 #include libc 头文件：

```
#include <hax.h>
#include <libc/string.h>    /* memcpy, strlen, strcmp ... */
#include <libc/stdlib.h>    /* atoi, malloc, free ...     */
#include <libc/stdio.h>     /* printf, fopen, fread ...   */
#include <libc/socket.h>    /* socket, connect, send ... */
```

完整列表见附录 A。

2.6 目录遍历（opendir / readdir）

HBOS libc 提供 POSIX 风格的目录遍历接口，声明在 `<libc/dirent.h>`。

函数	说明
`DIR *opendir(const char *path);`	打开目录，返回目录流；失败返回 NULL
`struct dirent *readdir(DIR *d);`	读取下一项，返回内部缓冲指针；无更多项返回 NULL
`int closedir(DIR *d);`	关闭目录流，释放资源
`int getdents(int fd, struct dirent *, unsigned);`	底层接口：从目录 fd 一次性读取目录项

`struct dirent` 关键字段：`d_name`（文件名，NUL 结尾）、`d_type`（`DT_DIR` 目录 / `DT_REG` 普通文件）、`d_ino`（inode 号）。

```
#include <libc/dirent.h>

DIR *d = opendir("/");
struct dirent *e;
while ((e = readdir(d)) != 0) {
    hax_printf("%s%s\n", e->d_name, e->d_type == DT_DIR ? "/" : "");
}
closedir(d);
```

> 实现细节：内核 `getdents` 单次调用即返回整个目录（不维护游标），`opendir` 因此一次性把目录项读入内部 16 KiB 缓冲，`readdir` 在其上迭代。完整示例见 4.4 节。

2.7 GUI 全屏画布接口

声明 `HAX_KIND_GUI`（或 `BOTH`）的应用可从图形桌面启动并**直接绘制到帧缓冲**，而非仅在终端输出文本。本接口为**全屏即时模式**画布：每帧「清屏 → 绘制 → 提交 → 取输入」，应用运行期间独占整屏（桌面让位），退出后桌面恢复。适合全屏小游戏。若想要可与桌面/其他应用**同时显示、可拖动的窗口**，请用 2.8 节的并发窗口接口。

函数	说明
`int hax_gui_begin(int *w, int *h);`	探测 GUI 是否可用；可用返回 1 并写入画布宽高，否则返回 0（应回退文本）
`void hax_gui_clear(HCOLOR c);`	用颜色（0xRRGGBB）填满整个画布
`void hax_gui_rect(int x,int y,int w,int h,HCOLOR c);`	填充矩形
`void hax_gui_text(int x,int y,const char*s,HCOLOR c,int scale);`	绘制文本（UTF-8，scale≥1 整数放大）
`void hax_gui_present(void);`	把画布提交到屏幕（绘制后必须调用才可见）
`int hax_gui_pollkey(void);`	轮询一个按键，返回键值；无按键返回 -1
`int hax_gui_pollmouse(int *x,int *y);`	轮询鼠标，写入绝对坐标，返回按键位掩码（bit0=左键）

另有一组**扩展绘图原语**（纯 SDK 实现，基于 `hax_gui_rect`，无额外系统调用）：

函数	说明
`void hax_gui_pixel(int x,int y,HCOLOR c);`	画一个像素
`void hax_gui_frame(int x,int y,int w,int h,int t,HCOLOR c);`	矩形描边（线宽 t）
`void hax_gui_line(int x0,int y0,int x1,int y1,HCOLOR c);`	直线（Bresenham）
`void hax_gui_fill_circle(int cx,int cy,int r,HCOLOR c);`	实心圆（水平 span 填充，高效）
`void hax_gui_circle(int cx,int cy,int r,HCOLOR c);`	圆环描边（中点画圆法）

```
int w, h;
if (!hax_gui_begin(&w, &h)) { hax_println("需要从桌面启动"); return 1; }
for (;;) {
    hax_gui_clear(0x202830);
    hax_gui_rect(20, 20, 120, 48, 0x14A6E0);
    hax_gui_text(28, 32, "Hello", 0xFFFFFF, 2);
    hax_gui_present();
    int k = hax_gui_pollkey();
    if (k == 'q' || k == 27) break;     /* q 或 ESC 退出 */
    hax_sleep(0);                        /* 让出 CPU */
}
```

> 全屏画布运行期间，桌面合成器自动让位给应用（应用通过 `hax_gui_begin` 登记为屏幕所有者）；应用退出后桌面立即恢复。若需与桌面共存的窗口，见 2.8。

2.8 并发窗口接口（推荐）

并发窗口接口让应用拥有一个**独立窗口**，与桌面、其他应用窗口**同时显示**，可拖动、可关闭。 每个窗口拥有自己的离屏表面，由桌面合成器每帧贴合到屏幕；应用与桌面在 100 Hz 抢占式调度下 **并发运行**（应用在后台持续刷新时，桌面时钟、其他窗口照常工作）。这是编写图形应用的推荐方式。

函数	说明
`int hax_win_open(const char*title,int w,int h);`	打开窗口（内容区 w×h），返回窗口 id（≥0）或 -1
`int hax_win_active(int*w,int*h);`	窗口是否仍活动：是返回 1 并写入当前 w、h；被关闭返回 0
`void hax_win_clear(HCOLOR c);`	用颜色填满窗口
`void hax_win_fill(int x,int y,int w,int h,HCOLOR c);`	窗口内填充矩形（坐标相对内容区）
`void hax_win_text(int x,int y,const char*s,HCOLOR c);`	窗口内绘制文本
`void hax_win_present(void);`	提交一帧（让出 CPU，使合成器尽快显示）
`int hax_win_poll(int*ev4);`	取事件到 ev[4]={type,a,b,c}，返回类型（HAX_EV_*，0=无）
`void hax_win_close(void);`	关闭并销毁窗口

事件类型：`HAX_EV_KEY`（ev[1]=键值）、`HAX_EV_MOUSE`（移动/按键； ev[1]=x ev[2]=y ev[3]=按键位，离开时 x=y=-1）、`HAX_EV_CLOSE`（用户点了关闭按钮，应退出）。

```
int w, h;
if (hax_win_open("我的应用", 360, 240) < 0) return 1;
while (hax_win_active(&w, &h)) {        /* 窗口被关闭时返回 0 */
    hax_win_clear(0x202830);
    hax_win_text(20, 20, "Hello", 0xFFFFFF);
    hax_win_present();
    int ev[4];
    int t = hax_win_poll(ev);
    if (t == HAX_EV_KEY && ev[1] == 'q') break;
    if (t == HAX_EV_CLOSE) break;
    hax_sleep(0);
}
hax_win_close();
```

实现：HIVE 合成器在桌面合成阶段把每个窗口表面贴到屏幕并加标题栏/边框；输入由桌面路由到聚焦 窗口的事件队列。应用退出后窗口自动回收。最多 8 个并发窗口，单窗口最大 900×640。窗口应用应从 **开始菜单**或在 GUI 终端用 `run` 启动（均为非阻塞）。鼠标按下后由内容窗口捕获， 移动和松开事件会继续投递；连续移动事件自动合并，防止事件队列被填满。

HIVE 0.1-beta5-gui.4 / Toolkit API 1.4 / 窗口 ABI v2：应用可用 `hive_window_query` 查询能力， 通过 `hive_window_create` 创建多个带显式代数句柄的窗口，并独立设置标题、几何和 普通/最小化/最大化状态。`hive_window_draw` 支持批量 Clear、Fill、Text 与 ARGB 位图上传，`hive_window_present_rect` 支持脏矩形提交；事件扩展为 Move、Resize、 Focus 与 State。旧 `hive_window_open` 单窗口接口保持兼容。

2.9 HIVE 标准窗口控件 API

HIVE（HBOS Interface & Visual Environment）是 HBOS 的桌面环境和 GUI 工具包。HAX 负责稳定的应用格式/系统调用 ABI；HIVE 完全运行在用户态，并在 `hax_win_*` 上提供控件、布局、主题和事件派发。新 GUI 应用首选 `#include <hive.h>`；旧 `hax_ui_*` 名称继续兼容。

控件	创建函数	交互
面板	`hive_ui_add_panel`	父子控件树、相对坐标、级联状态
标签	`hive_ui_add_label`	只读或动态文本
按钮	`hive_ui_add_button`	松开触发、Enter、Space → CLICK
文本框	`hive_ui_add_textbox`	输入、UTF-8 选区、鼠标拖选、Shift+左右、Ctrl+A
复选框	`hive_ui_add_checkbox`	点击或 Space 切换 → CHANGE
列表	`hive_ui_add_list`	点击、上下键、PageUp/PageDown → SELECT
进度条	`hive_ui_add_progress`	`hive_ui_set_value` 更新
滑杆	`hive_ui_add_slider`	拖动、方向键、Home/End → CHANGE
滚动条	`hive_ui_add_scrollbar`	横向/纵向拖动、方向键、PageUp/PageDown
菜单	`hive_ui_add_menu`	鼠标或键盘选择 → SELECT
图片	`hive_ui_add_image`	上传 0xAARRGGBB 位图
画布	`hive_ui_add_canvas`	应用回调自定义绘制
单选钮	`hive_ui_add_radio`	同一父容器内互斥，点击或 Space → CHANGE
开关	`hive_ui_add_toggle`	点击或 Space 切换 → CHANGE
下拉选择	`hive_ui_add_dropdown`	鼠标左右半区或方向键切换 → SELECT
数字微调	`hive_ui_add_spinbox`	加减按钮、方向键、Home/End → CHANGE
分隔线	`hive_ui_add_separator`	横向或纵向视觉分组

分组框	hive_ui_add_groupbox	带标题的父容器，可组织 Radio 组

当前 HIVE_API_MAJOR=1、HIVE_API_MINOR=4。单个 UI 最多 48 个控件，不使用动态内存；状态完全属于应用自己的 hive_ui_t。 主题统一提供普通、悬停、按下、禁用、选择和焦点环颜色。

函数	说明
hive_ui_init(ui)	初始化上下文和默认暗色主题
hive_layout_begin / hive_layout_row	建立带内边距和间距的纵向布局
hive_grid_cell(row,n,gap,i)	把一行等分成响应式网格
hive_ui_poll(ui,event)	读取窗口事件并派发到控件
hive_ui_draw(ui)	绘制全部可见控件和交互状态
hive_ui_set_enabled / set_visible	启用、禁用、显示或隐藏控件
hive_ui_set_text / set_value / get_value	更新或读取控件内容
hive_ui_set_parent / parent	建立或查询 Panel/Groupbox 父子关系，子控件使用相对坐标
hive_ui_set_rect / get_rect / remove	更新相对位置、读取窗口坐标或删除整棵子树
hive_textbox_select / select_all	按 UTF-8 码点边界设置或全选文本
hive_textbox_selection / copy_selection	查询选区范围或复制为 NUL 结尾 UTF-8 文本

```
#include <hive.h>
HIVE_APP("hello-ui", "最小HIVE 应用");

hive_ui_t ui;
hive_ui_init(&ui);
hive_ui_add_button(&ui, 1, hive_rect(20,20,120,36), "确定");

while (hive_window_active(&w, &h)) {
    hive_event_t ev;
    while (hive_ui_poll(&ui, &ev) != HIVE_EVENT_NONE) {
        if (ev.type == HIVE_EVENT_CLOSE) goto done;
        if (ev.type == HIVE_EVENT_CLICK && ev.widget_id == 1) {
            /* 执行动作 */
        }
    }
    hive_window_clear(ui.theme.window_bg);
    hive_ui_draw(&ui);
    hive_window_present();
    hive_yield();
}
done: hive_window_close();
```

HIVE Toolkit API 1.4 能按 UTF-8 码点移动、鼠标/键盘选择、替换和删除已有文本； 窗口键盘事件目前仍只直接产生可打印 ASCII，完整中文输入法、系统剪贴板和无障碍语义 将在后续版本追加。

2.10 TUI 终端控件 API

#include <hax.h> 随 hax_tui.h 提供一套文本终端控件 （**TUI Kit API 1.0**，HIVE_TUI_API_MAJOR/MINOR=1/0）。与窗口控件一样纯用户态、无新增系统调用、无全局变量、无动态内存：输出只依赖 stdout， 输入只依赖 stdin（hax_input 读整行），在内核控制台和图形终端中行为 一致。渲染刻意使用纯 ASCII（+ - | = [ ]），对齐按显示格计算 （CJK/全角字符占 2 格），截断不会切断 UTF-8 码点。

控件	函数	说明
标题分隔线	hax_tui_rule / hax_tui_hline	带标题或纯横线
进度条	hax_tui_progress	[====----] 45%；in_place 用 \r 原地刷新
边框盒	hax_tui_box	标题嵌顶边的 ASCII 盒
表格	hax_tui_table	列宽/对齐由 hax_tui_column_t 描述
菜单	hax_tui_menu	数字 1..N 或快捷字母选择，q 取消
确认	hax_tui_confirm	[Y/n] 默认值，回车取默认
输入行	hax_tui_input	带提示与默认值，UTF-8 安全截断
复选	hax_tui_checkbox	[x] 状态行 + y/n 切换

交互控件都是行驱动的；空行取默认值，q/Q 取消（返回 -1）， stdin 读到 EOF 同样返回 -1，因此即使被无输入管道启动也不会死循环。 解析逻辑（hax_tui_menu_parse / hax_tui_confirm_parse）与 渲染逻辑（*_to 系列）分离，便于宿主测试。

```c
#include <hive.h>
HAX_APP("svc", "服务管理", HAX_KIND_TUI);

static const char *const items[] = {"启动服务", "停止服务", "查看状态"};

int main(void) {
    hax_tui_rule("服务管理", 40);
    HI32 sel = hax_tui_menu("请选择操作", items, 3, NULL);
    if (sel < 0) return 0;                      /* q / EOF */
    hax_printf("选择了: %s\n", items[sel]);
    hax_tui_progress("执行", 100, 0, 100, 24, 1);
    hax_println("");
    return 0;
}
```

hax_tui_progress 的 in_place 模式依赖终端支持 \r 回车不换行；HBOS 图形终端会丢弃 \r，此时退化为逐行 输出（功能不受影响）。app/tui.c 是完整的控件展示应用。

第 3 章

RUN, DISCOVERY & DEBUG

3.1 TUI 与 GUI 类型说明

kind	标签	启动方式	输入输出
HAX_KIND_TUI	TUI	run <名>，或桌面终端窗口	stdin/stdout 重定向到终端
HAX_KIND_GUI	GUI	桌面启动器图标，或 run <名>	当前版本：输出到终端窗口
HAX_KIND_BOTH	TUI GUI	两种入口均可用	同上

GUI 应用绘制有三种方式：(1) 只输出文本（在终端窗口显示）；(2) **全屏画布** hax_gui_*（见 2.7，独占整屏，适合全屏游戏）；(3) **并发窗口** hax_win_*（见 2.8，独立窗口，与桌面/其他应用同时显示、可拖动，**推荐**）。后两者均直接绘制到帧缓冲。

3.2 apps / run 命令

```
HBOS> apps
catf      - 读取并显示文件内容                [TUI .hax]
leapyear  - 闰年判定                          [TUI GUI .hax]

HBOS> run leapyear
```

命令	作用
apps	列出所有已注册应用（内建命令 + .hax 应用），显示名称、描述、类型标签
run <名> [参数...]	运行应用；优先匹配内建命令，未命中后查找 .hax 表，再未命中报错

3.3 在图形桌面运行

在 HIVE 桌面中，打开“终端”窗口后即可像命令行一样输入 apps 与 run 命令。 此外，HAX_KIND_GUI（或 BOTH）类型的应用会**自动追加到开始菜单**（已固定应用网格的内置项之后）， 点击图标即可启动，等价于 run <名>。

3.4 参数传递

run myapp a b c 等价于 C 程序的 main(4, {"myapp","a","b","c",NULL})：

- argv[0] 始终为应用名（与 HAX_APP 中的 name 相同）。
- argv[1..argc-1] 为用户传入的额外参数，均为 NUL 结尾 UTF-8 字符串。
- argv[argc] 保证为 NULL（标准 POSIX 约定）。
- 参数最多 31 个（含 argv[0]），超出部分截断。

3.5 退出码约定

退出码	含义
0	成功
1	一般错误（参数错误、IO 失败等）
2	使用方法错误（参数数量不对）
127	命令未找到（由 shell/run 命令返回，非应用本身）
其他非零	应用自定义错误码

内核通过 task_wait 获取退出码并传回 run 命令；当前 shell 不打印退出码，可用 hax_printf 在退出前自行输出。

第 4 章

COMPLETE EXAMPLES

本章代码用于讲解 SDK，不作为预装应用打包。当前随系统保留的 HAX 应用只有 `catf` 与 `leapyear`。

4.1 最小应用（TUI，文档示例）

演示：`HAX_APP()`、文本输出、参数处理、`hax_pid()`。

```
/* myapp.c */
#include <hax.h>

HAX_APP("myapp", "最小HAX 应用", HAX_KIND_TUI);

int main(int argc, char **argv) {
    hax_println("你好，HBOS! 这是一个 .hax 应用。");
    hax_printf("我的PID 是 %d\n", hax_pid());

    if (argc > 1) {
        hax_print("收到参数:");
        for (int i = 1; i < argc; i++)
            hax_printf(" %s", argv[i]);
        hax_println("");
    }
    return 0;
}
```

把文件放入外部应用目录并构建后，可这样运行：

```
HBOS> run myapp 世界
你好，HBOS! 这是一个 .hax 应用。
我的PID 是 5
收到参数: 世界
```

4.2 输入循环（GUI，文档示例）

演示：`HAX_KIND_GUI`、循环输入、`atoi`、`hax_exit`。

```
/* input-loop.c */
#include <hax.h>
#include <libc/stdlib.h>    /* atoi */

HAX_APP("input-loop", "输入循环示例", HAX_KIND_GUI);

int main(int argc, char **argv) {
    (void)argc; (void)argv;

    /* 以PID 为简易随机种子，避免每次都一样 */
    int secret = (hax_pid() * 37 + 11) % 100 + 1;
    char line[32];

    hax_println("我想了一个 1-100 的整数，猜猜看! (输入q 退出)");

    for (int tries = 1; ; tries++) {
        hax_printf("第 %d 次猜测> ", tries);
        if (hax_input(line, sizeof(line)) < 0 || line[0] == 'q')
            break;

        int v = atoi(line);
        if (v < 1 || v > 100) {
            hax_println("请输入 1-100 之间的整数。");
            tries--;
            continue;
        }

        if      (v < secret) hax_println("太小了，再大一点。");
        else if (v > secret) hax_println("太大了，再小一点。");
        else {
            hax_printf("猜对了! 答案是 %d，你用了 %d 次。\n",
                       secret, tries);
            return 0;
        }
    }

    hax_printf("游戏结束。正确答案是 %d。\n", secret);
    return 0;
}
```

4.3 catf —— 读取并显示文件（TUI）

演示：参数处理、hax_read_file、标准 libc 调用（atoi）、错误处理与退出码。

```
/* app/catf.c */
#include <hax.h>

HAX_APP("catf", "读取并显示文件内容", HAX_KIND_TUI);

int main(int argc, char **argv) {
    if (argc < 2) {
        hax_println("用法: run catf <文件路径>");
        return 2;                       /* 用法错误 */
    }

    static char buf[8192];
    long n = hax_read_file(argv[1], buf, sizeof(buf) - 1);
    if (n < 0) {
        hax_printf("无法读取文件: %s\n", argv[1]);
        return 1;                       /* 一般错误 */
    }

    buf[n] = '\0';                      /* 当字符串处理 */
    hax_print(buf);
    if (n > 0 && buf[n - 1] != '\n')
        hax_println("");                /* 补一个换行 */

    hax_printf("\n[共 %ld 字节]\n", n);
    return 0;
}
```

```
HBOS> run catf /etc/hostname
hbos

[共 5 字节]
```

本示例只用了 hax_read_file。目录遍历见下一节。

4.4 ls —— 列出目录内容（TUI）

演示：opendir/readdir/closedir、d_type 判断、参数处理。

```
/* app/ls.c */
#include <hax.h>
#include <libc/dirent.h>

HAX_APP("ls", "列出目录下的文件与子目录", HAX_KIND_TUI);

int main(int argc, char **argv) {
    const char *path = (argc > 1) ? argv[1] : "/";

    DIR *d = opendir(path);
    if (!d) { hax_printf("无法打开目录: %s\n", path); return 1; }

    hax_printf("目录 %s:\n", path);
    struct dirent *ent;
    int files = 0, dirs = 0;
    while ((ent = readdir(d)) != 0) {
        if (ent->d_name[0] == '\0') continue;
        if (ent->d_type == DT_DIR) { hax_printf("  [DIR]  %s\n", ent->d_name); dirs++; }
        else                       { hax_printf("         %s\n", ent->d_name); files++; }
    }
    closedir(d);
    hax_printf("共 %d 个文件，%d 个目录。\n", files, dirs);
    return 0;
}
```

4.5 GUI 全屏画布（文档示例）

演示：hax_gui_begin 探测、鼠标轮询绘制、按键退出、无 GUI 时回退。 从图形桌面启动；按住左键涂鸦，c 清屏，q / ESC 退出。

```
/* canvas-example.c */
#include <hax.h>

HAX_APP("canvas-example", "GUI 全屏画布示例", HAX_KIND_GUI);

int main(int argc, char **argv) {
    (void)argc; (void)argv;
    int w, h;
    if (!hax_gui_begin(&w, &h)) {
        hax_println("此示例需要从图形桌面启动。");
        return 1;
    }
    hax_gui_clear(0x10141A);
    hax_gui_rect(0, 0, w, 28, 0x14A6E0);
    hax_gui_text(8, 6, "HBOS Paint  -  左键涂鸦 c 清屏 q 退出", 0xFFFFFF, 1);
    hax_gui_present();

    for (;;) {
        int mx, my;
        int btn = hax_gui_pollmouse(&mx, &my);
        if ((btn & 1) && my > 28) {           /* 左键拖动绘制 */
            hax_gui_rect(mx - 4, my - 4, 8, 8, 0xF5C842);
            hax_gui_present();
        }
        int k = hax_gui_pollkey();
        if (k == 'q' || k == 27) break;
        hax_sleep(0);
    }
    return 0;
}
```

4.6 HIVE 并发窗口（文档示例）

以下片段演示按钮、复选框、 滑杆、进度条、响应式布局及完整键盘操作。

```
/* window-example.c（节选） */
#include <hive.h>
HIVE_APP("window-example", "HIVE 并发窗口示例");

int main(int argc, char **argv) {
    hive_ui_t ui;
    hive_ui_init(&ui);
    hive_ui_add_checkbox(&ui, ID_AUTO, hive_rect(20,80,140,28),
                         "自动计数", 1);
    hive_ui_add_slider(&ui, ID_SPEED, hive_rect(180,80,140,28),
                       1, 10, 6, 1);
    hive_ui_add_button(&ui, ID_ADD, hive_rect(20,140,140,36), "增加 10");

    while (hive_window_active(&w, &h)) {
        hive_event_t ev;
        while (hive_ui_poll(&ui, &ev) != HIVE_EVENT_NONE) {
            if (ev.type == HIVE_EVENT_CLOSE) goto done;
            if (ev.type == HIVE_EVENT_CLICK && ev.widget_id == ID_ADD)
                count += 10;
        }
        hive_window_clear(ui.theme.window_bg);
        hive_ui_draw(&ui);
        hive_window_present();
        hive_yield();
    }
done:
    hive_window_close();
    return 0;
}
```

4.7 HIVE Toolkit API 1.4 控件（文档示例）

以下片段覆盖响应式布局，以及 1.4 新增的分组框、单选钮、开关、下拉选择、 数字微调和分隔线。Radio 通过共同父容器形成互斥组：

```
hive_layout_t layout;
hive_layout_begin(&layout, hive_rect(0,0,width,height), 20, 10);

hive_rect_t form = hive_layout_row(&layout, 32);
hive_ui_widget(&ui, ID_NAME)->rect =
    hive_grid_cell(form, 3, 12, 0);
hive_ui_widget(&ui, ID_ENABLED)->rect =
    hive_grid_cell(form, 3, 12, 2);

hive_ui_add_slider(&ui, ID_SLIDER, hive_layout_row(&layout, 28),
                   0, 100, 35, 5);

static const char *const scales[] = {"100%", "125%", "150%"};
hive_ui_add_groupbox(&ui, ID_GROUP, hive_rect(20,160,300,90), "渲染模式");
hive_ui_add_radio(&ui, ID_BASIC, hive_rect(32,182,120,22), "标准", 1);
hive_ui_add_radio(&ui, ID_PRO, hive_rect(32,210,120,22), "增强", 0);
hive_ui_set_parent(&ui, ID_BASIC, ID_GROUP);
hive_ui_set_parent(&ui, ID_PRO, ID_GROUP);
hive_ui_add_toggle(&ui, ID_AUTO, hive_rect(180,182,120,22), "自动", 1);
hive_ui_add_dropdown(&ui, ID_SCALE, hive_rect(20,270,140,28), scales,3,0);
hive_ui_add_spinbox(&ui, ID_COUNT, hive_rect(180,270,120,28), 0,100,10,5);
hive_ui_add_separator(&ui, ID_SEP, hive_rect(20,310,280,4), 0);

while (hive_window_active(&width, &height)) {
    hive_event_t ev;
    while (hive_ui_poll(&ui, &ev) != HIVE_EVENT_NONE) {
        if (ev.type == HIVE_EVENT_CHANGE &&
            ev.widget_id == ID_SLIDER)
            hive_ui_set_value(&ui, ID_PROGRESS, ev.value);
    }
    hive_window_clear(ui.theme.window_bg);
    hive_ui_draw(&ui);
    hive_window_present();
    hive_yield();
}
```

提示：这些片段只用于 API 讲解；如需试验，请放入外部应用目录构建。

4.8 实机运行画面（QEMU 截图）

以下画面来自 HBOS `v0.1-beta5-pre6` 在 QEMU（q35、1024M、1920×1080）中的真实运行截图，展示 HIVE 桌面、开始菜单（应用启动器，含 `widgets` 控件展示应用）与 TUI Kit 演示（`tui`，在实机文本控制台运行）。

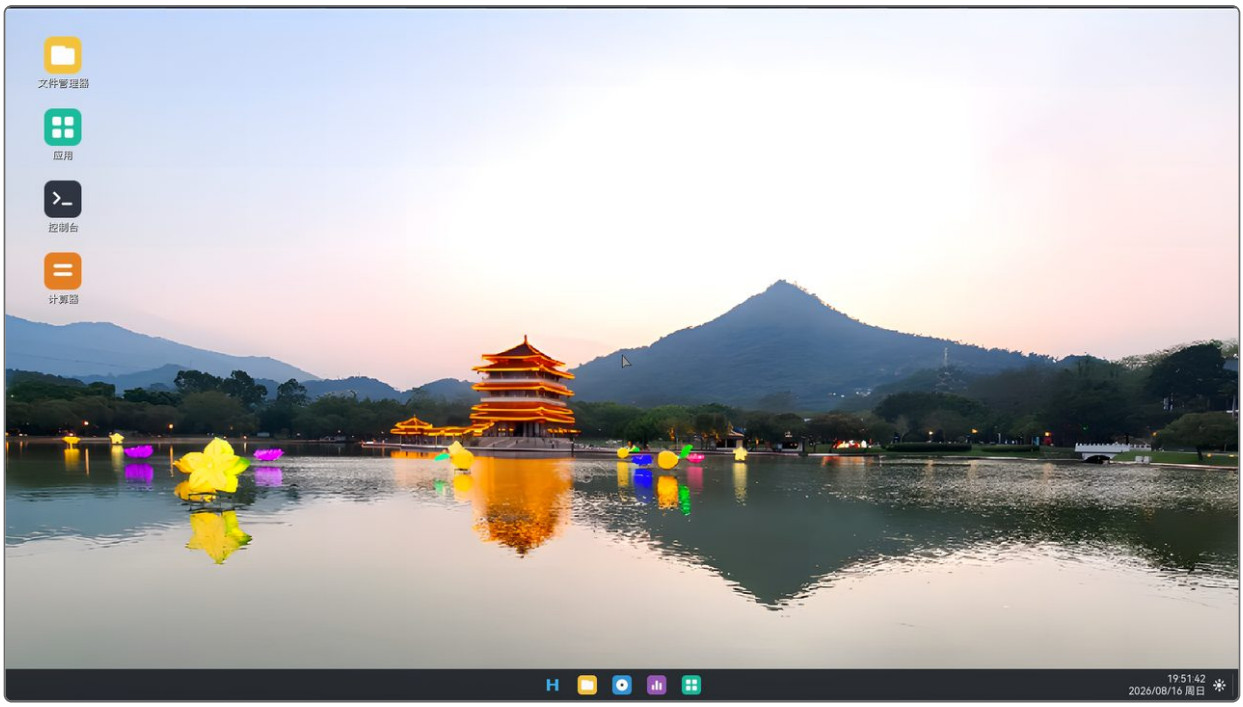


图 4-1：HIVE 桌面（任务栏、桌面图标与壁纸）



图 4-2：开始菜单——应用启动器（搜索框 + 应用网格，含控件展示）



图 4-3：TUI Kit 1.0 演示在实机文本控制台运行（表格、边框盒、进度条、菜单）

附录 A

POSIX SYSCALL QUICK REFERENCE

除 SDK 外，HAX 应用可直接使用 HBOS 用户态 libc 暴露的 POSIX 接口（声明在 `src/user/libc/` 下各头文件）。HBOS 当前公开 96 个系统调用，覆盖以下类别：

类别	接口（常用子集）
文件 I/O	`open close read write lseek fstat stat unlink rename`
进程控制	`getpid getppid exit sleep usleep fork execve waitpid kill`
文件系统	`mkdir rmdir getcwd chdir access ftruncate readlink opendir readdir closedir getdents`
内存管理	`sbrk brk mmap munmap mprotect malloc free realloc calloc`
网络	`socket bind listen accept connect send recv setsockopt`
标准 I/O	`printf fopen fclose fread fwrite fgets fputs fseek ftell`
字符串	`strlen strcpy strncpy strcmp strncmp strcat strchr strstr memcpy memset memmove`

转换	atoi atol strtol strtoul itoa sprintf snprintf

网络 DNS：HBOS 在网络配置未提供 DNS 地址时自动回退公共解析器（8.8.8.8），因此即便"静态 IP 未填 DNS"，connect 域名参数与高层 dns_resolve 接口仍可正常工作。

附录 B

hax_meta_t BINARY LAYOUT

下表为 hax_meta_t 在 .haxmeta 段中的精确二进制布局，供需要手工构造或解析元数据的工具开发者参考。

偏移（字节）	大小	字段	说明
0	4	magic	固定值 0x4D584148（小端），ASCII HAXM
4	4	kind	1=TUI，2=GUI，3=BOTH
8	32	name[32]	NUL 结尾 UTF-8 应用名，不足 32 字节用 NUL 补齐
40	64	desc[64]	NUL 结尾 UTF-8 描述，不足 64 字节用 NUL 补齐

总大小：**104 字节**，结构体无填充（__attribute__((packed)) 默认无需指定，字段已自然对齐）。

验证命令（宿主 Linux）：

```
readelf -S build/app/leapyear.hax | grep haxmeta
# 应看到 .haxmeta  PROGBITS  ... 104 字节

python3 - <<'EOF'
import struct, sys
data = open("build/app/leapyear.hax","rb").read()
# 搜索 magic
idx = data.find(b'HAXM')
if idx < 0: print("无 .haxmeta"); sys.exit(1)
magic, kind = struct.unpack_from("<II", data, idx)
name = data[idx+8:idx+40].rstrip(b'\x00').decode()
desc = data[idx+40:idx+104].rstrip(b'\x00').decode()
print(f"kind={kind} name={name!r} desc={desc!r}")
EOF
```

附录 C

TROUBLESHOOTING

以下为构建和运行 HAX 应用时的常见问题及解决方法。

症状	可能原因	解决方法
hive_window_open() 返回 -1	应用运行在 no-GUI 版本，或 HIVE 尚未启动	从完整 HIVE 桌面启动；no-GUI 构建会保留 HAX ABI 但明确拒绝创建窗口
genhax.py: magic mismatch	手写的 .hax 缺少合法 .haxmeta 段	检查 HAX_APP() 宏是否在全局作用域；或接受 genhax.py 的文件名兜底行为
run myapp: not found	应用名与 HAX_APP 第一参数不一致，或 make 未重新打包	确认 name 参数；执行 make 后重建 ISO 并重启
应用运行后立即退出（无输出）	ELF 加载失败（地址冲突、段大小溢出）或 main 未被链接进去	用 readelf -l build/app/xxx.hax 检查 PT_LOAD 段；确认 crt0 + main 均链接
ld: cannot find -lhax	使用了 -lhax 链接标志（HAX SDK 不是共享库）	删除 -lhax；SDK 通过 #include <hax.h> 内联或编译进 libc 目标文件
Make grouped target 报错	GNU Make 版本 < 4.3（&: 语法不支持）	make --version 确认版本；Ubuntu 22.04+ 自带 Make 4.3+